

目录

MaixCAM	MaixPy	摄像头使用 - MaixPy	• • • • •	2
MaixPy	MaixCAM	触摸屏使用方法 - MaixPy	• • • • •	10
MaixCAM	MaixPy	屏幕使用 - MaixPy	• • • • •	15
MaixCAM	MaixPy	图像基础操作 - MaixPy	• • • • •	21
MaixCAM	MaixPy	使用 OpenCV - MaixPy	• • • • •	34
MaixCAM	MaixPy	寻找直线 - MaixPy	• • • • •	41
MaixCAM	MaixPy	寻找色块 - MaixPy	• • • • •	51
MaixCAM	MaixPy	二维码识别 - MaixPy	• • • • •	60
MaixCAM	MaixPy	识别 Apriltag 标签 - MaixPy	• • • • •	66
MaixCAM	MaixPy	条形码识别 - MaixPy	• • • • •	79

MaixCAM MaixPy 摄像头使用

2024-10-24

更新历史

日期	版本	作者	更新内容
2024-10-24	1.1.0	neucrack	增加 USB 摄像头支持说明
2024-08-21	1.0.1	YWJ	修正文档部分bug, 增加部分内容
2024-04-03	1.0.0	neucrack	初版文档

一、简介

对于 MaixCAM 默认搭载了 GC4653 摄像头，或者可选的 OS04A10 摄像头或者全局快门摄像头，甚至是 HDMI 转 MIPI 模块，都可以直接用简单的 API 调用。

二、API 文档

本文介绍常用方法，更多 API 使用参考 maix.camera 模块的文档。

三、摄像头切换

目前支持的摄像头：

- **GC4653**：M12 通用镜头，1/3" 传感器，画质清晰，4M 像素。
- **OS04A10**：M12 通用镜头，1/1.8" 大底传感器，画质超清，4M像素。
- **OV2685**：不支持镜头更换，1/5"传感器，2M 像素，画质最差，成本最低，一般不建议使用。
- **SC035HGS**：黑白全局快门摄像头，30W黑白像素，适合拍摄高速物体。

系统会自动切换，只接硬件换上即可使用。

四、获取摄像头的图像信息

使用 MaixPy 轻松获取：

```
1 | from maix import camera
2 |
3 | cam = camera.Camera(640, 480)
4 |
5 | while 1:
6 |     img = cam.read()
7 |     print(img)
```

这里我们从 `maix` 模块导入 `camera` 模块，然后创建一个 `Camera` 对象，指定图像的宽度和高度。然后在一个循环中不断读取图像，默认出的图为 `RGB` 格式，如果需要 `BGR` 格式，其它格式请看 API 文档。

你还可以获取灰度图像

```
1 | from maix import camera, image
2 | cam = camera.Camera(640, 480,
   | image.Format.FMT_GRAYSCALE) # 设置
```

还可以获取NV21图像

```
1 | from maix import camera, image
2 | cam = camera.Camera(640, 480,
    image.Format.FMT_YVU420SP) # 设置输出
    NV21图像
```

注意：如果设置了很高的分辨率（例如 2560x1440）时需要关闭MaixVision的在线浏览功能，否则可能会因为内存不足导致代码运行异常。

五、设置摄像头的帧率

GC4653最大支持 2560x1440 30fps、1280x720 60fps 和 1280x720 80fps 三种配置，由创建 Camera 对象时传入的 width，height，fps 参数来选择帧率。

OS04A10最大支持 2560x1440 30fps、1280x720 90fps 两种配置，其中 1280x720 是基于 2560x1440 居中裁剪的画面

5.1. 设置帧率为30帧

```
1 | from maix import camera
2 | cam = camera.Camera(640, 480,
3 | fps=30)          # 设置帧率为30帧
4 | # or
    cam = camera.Camera(1920, 1280)
    # 分辨率高于1280x720时帧率会设置为30帧
```

5.2. 设置帧率为60帧

```
1 | from maix import camera
2 | cam = camera.Camera(640, 480,
3 | fps=60)          # 设置帧率为60帧
4 | # or
   | cam = camera.Camera(640, 480)
   | # 分辨率低于或等于1280x720时帧率会设置
   | 为80fps
```

5.3. 设置帧率为80帧

```
1 | from maix import camera
2 | cam = camera.Camera(640, 480,
   | fps=80)          # 设置帧率为80帧
```

注意：

1. 如果 `Camera` 传入的尺寸大于 `1280x720`，例如写成 `camera.Camera(1920, 1080, fps=60)`，此时 `fps` 参数将会失效，帧率将保持在 `30fps`。
2. `60/80fps` 与 `30fps` 的画面相比会有几个像素的偏移，在对视角有严格要求的应用下需要注意修正偏移。
3. 需要注意由于 `60/80fps` 和 `30fps` 共用了 `isp` 配置，在某些环境下两种帧率下的画面画质会存在一些偏差。
4. 摄像头要看体制，有些体制无法设置到80fps，会出现画面有奇怪的纹路，请换会正常的 60fps使用。

六、图像矫正

对于画面存在鱼眼等畸变的情况，可以使用 `Image` 对象下的 `lens_corr` 函数对图片进行畸变矫正。一般情况只需要调大和调小 `strength` 的值来将画面调整到合适效果即可。

```

1  from maix import camera,
2  display, app, time
3
4  cam = camera.Camera(320, 240)
5  disp = display.Display()
6  while not app.need_exit():
7      t = time.ticks_ms()
8      img = cam.read()
9      img =
10 img.lens_corr(strength=1.5)    # 调整
    strength的值直到画面不再畸变
    disp.show(img)

```

注意由于是软件矫正，需要耗费一定时间，另外也可以只接用无畸变镜头（询问商家）从硬件层面解决。

七、跳过开头的帧

摄像头初始化的一小段时间，可能图像采集还没稳定出现奇怪的画面，可以通过 `skip_frames` 函数跳过开头的几帧：

```

1  cam = camera.Camera(640, 480)
2  cam.skip_frames(30)          # 跳过
    开头的30帧

```

八、显示摄像头获取的图像

MaixPy 提供了 `display` 模块，可以方便的显示图像：

```

1  from maix import camera, display
2
3  cam = camera.Camera(640, 480)
4  disp = display.Display()

```

```
5  
6 while 1:  
7     img = cam.read()  
8     disp.show(img)
```

九、设置摄像头参数

9.1. 设置曝光时间

注意设置曝光时间后，摄像头会切换到手动曝光模式，如果要切换回自动曝光模式需运行 `cam.exp_mode(0)`

```
1 cam = camera.Camera()  
2 cam.exposure(1000)
```

9.2. 设置增益

注意设置增益后，摄像头会切换到手动曝光模式，如果要切换回自动曝光模式需运行 `cam.exp_mode(0)`。自定义的增益值只能在手动曝光模式下生效。

```
1 cam = camera.Camera()  
2 cam.gain(100)
```

9.3. 设置白平衡

```
1 cam = camera.Camera()  
2 cam.awb_mode(1) # 0, 开启白平衡; 1, 关闭白平衡
```

9.4. 设置亮度、对比度和饱和度

```
1 cam = camera.Camera()  
2 cam.luma(50) # 设置亮度，范
```

```
3 | 围[0, 100]
4 | cam.constrast(50)      # 设置对比度,
   | 范围[0, 100]
   | cam.saturation(50)    # 设置饱和度,
   | 范围[0, 100]
```

9.5. 更改图片长宽

```
1 | cam = camera.Camera(width=640,
   | height=480)
```

或

```
1 | cam = camera.Camera()
2 | cam.set_resolution(width=640,
   | height=480)
```

9.6. 读取原始raw图

注意输出的 `raw` 图是原始的 `bayer` 图，并且不同摄像头模组输出的 `bayer` 图格式可能不一样。

```
1 | cam = camera.Camera(raw=true)
2 | raw_img = cam.read_raw()
3 | print(raw_img)
```

如果需要在第三方软件打开 `raw` 图，需要额外在PC端进行转换，可以参考 `bayer_to_tiff` 的示例代码

十、使用 USB 摄像头

除了使用开发板自带的 MIPI 接口摄像头，你也可以使用 USB 外接 USB 摄像头。

方法：

- 先在开发板设置里面 **USB设置** 中选择 **USB 模式** 为 **HOST** 模式。如果没有屏幕，可以用 `examples/tools/maixcam_switch_usb_mode.py` 脚本进行设置。
- `maix.camera` 模块目前(2024.10.24) 还不支持 USB 摄像头，不过你可以参考 **OpenCV 使用 USB 摄像头**。



MaixPy / MaixCAM 触摸屏使用方法

一、简介

对于 MaixCAM 自带了一个触摸屏，写应用时配合触摸屏可以实现很多有趣应用，我们可以通过 API 读取到触摸屏的点按操作。

二、MaixPy 读取触摸

MaixPy 提供了一个简单的

`maix.touchscreen.TouchScreen` 类来读取，举例：

```
1  from maix import touchscreen, appCopy
2  time
3
4  ts = touchscreen.TouchScreen()
5
6  pressed_already = False
7  last_x = 0
8  last_y = 0
9  last_pressed = False
10 while not app.need_exit():
11     x, y, pressed = ts.read()
12     if x != last_x or y != last_y or
13     pressed != last_pressed:
14         print(x, y, pressed)
15         last_x = x
16         last_y = y
17         last_pressed = pressed
18     if pressed:
19
```

```

19         pressed_already = True
20     else:
21         if pressed_already:
22             print(f"clicked, x: {x},
y: {y}")
                pressed_already = False
                time.sleep_ms(1) # sleep some
time to free some CPU usage

```

三、配合屏幕实现交互

配合屏幕可以做出一些用户交互的内容，更多可以看 [MaixPy/examples/vision/touchscreen](#) 目录下例程。

如前面的文章介绍的，我们要往屏幕显示内容，一般是得到一个 `maix.image.Image` 对象，然后调用

`disp.show(img)` 来显示这张图像。

实现一个按钮的最原始和简单的方法就是在这个图像上画一个按钮，然后判断用户触摸到这个区域就算是触发了按下事件，注意图像的大小要和屏幕的大小保持一致：

```

1  from maix import touchscreen, app,
2  time, display, image
3
4  ts = touchscreen.TouchScreen()
5  disp = display.Display()
6
7  img = image.Image(disp.width(),
8  disp.height())
9
10 # draw exit button
11 exit_label = "< Exit"
12 size = image.string_size(exit_label)
13 exit_btn_pos = [0, 0, 8*2 +
14 size.width(), 12 * 2 +
15 size.height()]
16 img.draw_string(8, 12, exit_label,
17 image.COLOR_WHITE)
18 img.draw_rect(exit_btn_pos[0],

```

```

19 exit_btn_pos[1], exit_btn_pos[2],
20 exit_btn_pos[3], image.COLOR_WHITE,
21 2)
22
23 def is_in_button(x, y, btn_pos):
    return x > btn_pos[0] and x <
    btn_pos[0] + btn_pos[2] and y >
    btn_pos[1] and y < btn_pos[1] +
    btn_pos[3]

    while not app.need_exit():
        x, y, pressed = ts.read()
        if is_in_button(x, y,
            exit_btn_pos):
            app.set_exit_flag(True)
            img.draw_circle(x, y, 1,
                image.Color.from_rgb(255, 255, 255),
                2)
            disp.show(img)

```

四、屏幕和图像大小不一样时如何处理

上面的例子可以看到 `img` 大小和屏幕大小一样，如果你的 `img` 和屏幕大小不一样怎么办（比如上面使用 `img = image.Image(240, 240)`，比如屏幕是 `640x480`，图像是 `240x240`，`disp.show(img)` 的默认行为是 `image.Fit.FIT_CONTAIN`，即把图片放大到 `480x480` 然后边上填充黑色，如果你在 `240x240` 的图上画了按钮，比如坐标 `(0, 0, 60, 40)`，那么按钮也会自动被放大，所以触摸判断的坐标就不能用 `(0, 0, 60, 40)`，需要用 `((640 - 480) / 2, 0, 480/240*60, 480/240*40)`，即 `(80, 0, 120, 80)`。

这里为了方便缩放图像时，快速计算源图像的点或者矩形框在缩放后的目标图像的位置和大小，提供了

`image.resize_map_pos` 函数来进行此计算过程。

```
from maix import touchscreen, app,
time, display, image

ts = touchscreen.TouchScreen()
disp = display.Display()

img = image.Image(240, 240)
img.draw_rect(0, 0, img.width(),
img.height(), image.COLOR_WHITE)

# draw exit button
exit_label = "< Exit"
size = image.string_size(exit_label)
exit_btn_pos = [0, 0, 8*2 +
size.width(), 12 * 2 +
size.height()]
img.draw_string(8, 12, exit_label,
image.COLOR_WHITE)
img.draw_rect(exit_btn_pos[0],
exit_btn_pos[1], exit_btn_pos[2],
exit_btn_pos[3], image.COLOR_WHITE,
2)
# 图像按键坐标映射到屏幕上的坐标
exit_btn_disp_pos =
image.resize_map_pos(img.width(),
img.height(), disp.width(),
disp.height(),
image.Fit.FIT_CONTAIN,
exit_btn_pos[0], exit_btn_pos[1],
exit_btn_pos[2], exit_btn_pos[3])

def is_in_button(x, y, btn_pos):
    return x > btn_pos[0] and x <
btn_pos[0] + btn_pos[2] and y >
btn_pos[1] and y < btn_pos[1] +
btn_pos[3]

while not app.need_exit():
    x, y, pressed = ts.read()
    if is_in_button(x, y,
```

```
exit_btn_disp_pos):
    app.set_exit_flag(True)
    # 屏幕的坐标映射回图像上对应的坐标，
    然后在图像上画点
    x, y =
image.resize_map_pos_reverse(img.width
img.height(), disp.width(),
disp.height(),
image.Fit.FIT_CONTAIN, x, y)
    img.draw_circle(x, y, 1,
image.Color.from_rgb(255, 255, 255),
2)
    disp.show(img,
fit=image.Fit.FIT_CONTAIN)
```



MaixCAM MaixPy 屏幕使用

2024-03-31

▼ 更新历史

日期	版本	作者	更新内容
2024-03-31	1.0.0	neucrack	初版文档

一、简介

MaixPy 提供了 `display` 模块，可以将图像显示到屏幕上，同时，也可以将图像发送到 MaixVision 显示，方便调试和开发。

二、API 文档

本文介绍常用方法，更多 API 请看 API 文档的 `display` 部分。

三、使用屏幕

- 导入 `display` 模块：

```
1 | from maix import display
```

- 创建一个 `Display` 对象：

```
1 | disp = display.Display()
```

- 显示图像:

```
1 | disp.show(img)
```

这里 `img` 对象是 `maix.image.Image` 对象, 可以通过 `camera` 模块的 `read` 方法获取, 也可以通过 `image` 模块的 `load` 方法加载文件系统中的图像, 也可以通过 `image` 模块的 `Image` 类创建一个空白图像。

比如:

```
1 | from maix import image, display
2 |
3 | disp = display.Display()
4 | img = image.load("/root/dog.jpg")
5 | disp.show(img)
```

这里需要先把 `dog.jpg` 文件传到设备的 `/root` 目录下。

显示文字:

```
1 | from maix import image, display
2 |
3 | disp = display.Display()
4 | img = image.Image(320, 240)
5 | img.draw_rect(0, 0, disp.width(),
6 | disp.height(),
7 | color=image.Color.from_rgb(255, 0,
8 | 0), thickness=-1)
   | img.draw_rect(10, 10, 100, 100,
   | color=image.Color.from_rgb(255, 0,
   | 0))
   | img.draw_string(10, 10, "Hello
   | MaixPy!",
   | color=image.Color.from_rgb(255, 255,
   | 255))
```



```
disp.show(img)
```

从摄像头读取图像并显示：

```
1 | from maix import camera, display,  
2 | app  
3 |  
4 | disp = display.Display()  
5 | cam = camera.Camera(320, 240)  
6 | while not app.need_exit():  
7 |     img = cam.read()  
    disp.show(img)
```

这里用了一个 `while not app.need_exit():` 是方便程序在其它地方调用 `app.set_exit_flag()` 方法后退出循环。

四、调整背光亮度

在系统的 `设置` 应用中可以手动调整背光亮度，如果你想在程序中调整背光亮度，可以使用 `set_backlight` 方法，参数就是亮度百分比，取值范围是 0-100：

```
1 | disp.set_backlight(50)
```

注意，程序退出回到应用选择界面后会自动恢复到系统设置的背光亮度。

如果亮度设置到 `100%` 仍然觉得暗，可以尝试修改 `/boot/board` 文件中的 `disp_max_backlight=50` 选项为更大的值，当 `disp_max_backlight=100` 并且 `disp.set_backlight(100)` 时硬件上背光控制引脚输出 `100%` 占空比即高电平。即最终输出到硬件的占空比

`= set_backlight 设置值 * disp_max_backlight`。

注意，修改最大亮度限制会带来功耗和发热量的上升，按照自己实际需求合理设置，不要盲目追求拉满亮度。

五、显示到 MaixVision

在使用 MaixVision 运行代码时，能够将图像显示到 MaixVision 上，方便调试和开发。

在调用 `show` 方法时，会自动压缩图像并发送到 MaixVision 显示。

当然，如果你没有屏幕，或者为了节省内存不想初始化屏幕，也可以直接调用 `maix.dispaly` 对象的 `send_to_maixvision` 方法发送图像到 MaixVision 显示。

```
1  from maix import image,display
2
3  img = image.Image(320, 240)
4  disp = display.Display()
5
6  img.draw_rect(0, 0, img.width(),
7  img.height(),
8  color=image.Color.from_rgb(255, 0,
9  0), thickness=-1)
10 img.draw_rect(10, 10, 100, 100,
11 color=image.Color.from_rgb(255, 0,
12 0))
13 img.draw_string(10, 10, "Hello
14 MaixPy!",
15 color=image.Color.from_rgb(255, 255,
16 255))
17 display.send_to_maixvision(img)
```

六、更换其它型号屏幕

如果想换不同尺寸的屏幕，可以到 [商城](#) 咨询购买。

对于 MaixCAM，目前支持 4 款屏幕和 1 款 MIPI 转 HDMI 模块：

- 2.3寸 552x368 分辨率电容触摸屏：MaixCAM 带的屏幕。
- 2.4寸 640x480 分辨率电容触摸屏：MaixCAM-Pro 带的屏幕。
- 5寸 854x480 分辨率无触摸屏：注意无触摸，类似手机屏幕大小。
- 7寸 1280x800 分辨率电容触摸屏：7寸大屏，适合更多需要固定屏幕观看场景。
- LT9611（MIPI转HDMI模块）支持1280x720等多种分辨率，适合驱动各种HDMI屏。

不同屏幕的刷新图像时间差别在1~5毫秒，差别不是很大，主要的区别在于图像分辨率大了图像处理时间的差别。

更换屏幕需要同时**修改配置文件**，否则可能刷新时序不同会导致**烧屏**（屏幕留下显示过的影子），所以需要注意，最好严格按照下面的步骤操作，如果出现了烧屏的问题也不要紧张，断电放置一晚上一般会恢复。

- 按照烧录系统的文档烧录系统，烧录完成后会有 U 盘出现。
- 打开 U 盘内容，看到有一个 `board` 文件。
- 编辑 `board` 文件，修改 `panel` 键值，取值如下：
 - 2.3寸（MaixCAM 自带屏幕）：
`st7701_hd228001c31`。
 - 2.4寸（MaixCAM-Pro 自带屏幕）：
`st7701_lct024bsi20`。
 - 5寸：`st7701_dxq5d0019_v0` 早期（2023年）测试屏幕 `st7701_dxq5d0019b480854`。

- 7寸: `mt970920b` , 早期 (2023年) 测试屏幕用 `zct2133v1` 。
- LT9611 (MIPI转HDMI模块) :
 - 接线:
 - LT9611 I2C <---> MaixCAM I2C5
 - LT9611 MIPI IN <---> MaixCAM MIPI OUT
 - 支持的配置
 - `lt9611_1280x720_60hz` : 1280x720 60Hz
 - `lt9611_1024x768_60hz` : 1024x768 60Hz
 - `lt9611_640x480_60hz` : 640x480 60Hz
 - `lt9611_552x368_60hz` : 552x368 60Hz
- 保存 `board` , 并且**点击弹出 U 盘**, 不要直接断电, 否则可能文件丢失。
- 按下板子的 `reset` 按键, 或者重新上电启动。

以上的方式最保险, 保证上电前已经设置好了屏幕型号, 如果你已经烧录好系统了, 也可以修改系统的 `/boot/board` 文件然后重启。

早期的系统和二进制应用(< 2024.11.25)依赖的是 `uEnv.txt` 里面的 `panel` 键值, 如果系统和应用比较老旧, 修改了 `board` 也可以同时将 `uEnv.txt` 中一同修改。

MaixCAM MaixPy 图像基础操作

2024-07-08

▼ 更新历史

日期	版本	作者	更新内容
2024-07-08	1.1.0	neucrack	优化 cv 和 image 转换文档
2024-04-03	1.0.0	neucrack	初版文档

一、简介

视觉应用中图像占据非常重要的位置，不管是图片还是视频，因为视频本质上就是一帧帧的图像，所以图像处理是视觉应用的基础。

二、API 文档

本文介绍常用方法，更多 API 参考 [maix.image](#) 模块的文档。

三、图像格式

MaixPy 提供基础图像模块 `image`，里面最重要的就是 `image.Image` 类，用于图像的创建以及各种图像基础操

作，以及图像加载和保存等。

图像格式有很多，一般我们用 `image.Format.FMT_RGB888` 或者 `image.Format.FMT_RGBA8888` 或者 `image.Format.FMT_GRAYSCALE` 或者 `image.Format.FMT_BGR888` 等。

大家知道 `RGB` 三色可以合成任意颜色，所以一般情况下我们使用 `image.Format.FMT_RGB888` 就足够，`RGB888` 在内存中是 `RGB packed` 排列，即在内存中的排列：

像素1_红色，像素1_绿色，像素1_蓝色，像素2_红色，像素2_绿色，像素2_蓝色，... 依次排列。

四、创建图像

创建图像很简单，只需要指定图像的宽度和高度以及图像格式即可：

```
1 | from maix import image
2 |
3 | img = image.Image(320, 240,
4 | image.Format.FMT_RGB888)
5 | print(img)
   | print(img.width(), img.height(),
   | img.format())
```

`320` 是图像的宽度，`240` 是图像的高度，`image.Format.FMT_RGB888` 是图像的格式，格式参数可以省略，默认是 `image.Format.FMT_RGB888`。

这里通过 `img.width()`、`img.height()`、`img.format()` 可以获取图像的宽度、高度和格式。

五、显示到屏幕

MaixPy 提供了 `maix.display.Display` 类，可以方便的显示图像：

```
1 from maix import image, display
2
3 disp = display.Display()
4
5 img = image.Image(320, 240,
6 image.Format.FMT_RGB888)
disp.show(img)
```

注意这里因为没有图像数据，所以显示的是黑色的图像，修改画面看后文。

六、从文件系统读取图像

MaixPy 提供了 `maix.image.load` 方法，可以从文件系统读取图像：

```
1 from maix import image
2
3 img = image.load("/root/image.jpg")
4 if img is None:
5     raise Exception(f"load image
6 failed")
print(img)
```

注意这里 `/root/image.jpg` 是提前传输到了板子上的，方法可以看前面的教程。

可以支持 `jpg` 和 `png` 格式的图像。

七、保存图像到文件系统

MaixPy 的 `maix.image.Image` 提供了 `save` 方法，可以保存图像到文件系统：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5
6 # do something with img
img.save("/root/image.jpg")
```

八、画框

`image.Image` 提供了 `draw_rect` 方法，可以在图像上画框：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
img.draw_rect(10, 10, 100, 100,
image.Color.from_rgb(255, 0, 0))
```

这里的参数依次是：`x`，`y`，`w`，`h`，`color`，`x` 和 `y` 是框的左上角坐标，`w` 和 `h` 是框的宽度和高度，`color` 是框的颜色，可以使用 `image.Color.from_rgb` 方法创建颜色。

可以用 `thickness` 指定框的线宽，默认是 `1`，

也可以画实心框，传参 `thickness=-1` 即可：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
img.draw_rect(10, 10, 100, 100,
(255, 0, 0), thickness=-1)
```

九、写字符串

`image.Image` 提供了 `draw_string` 方法，可以在图像上写字：

```
1 | from maix import image
2 |
3 | img = image.Image(320, 240,
4 | image.Format.FMT_RGB888)
   img.draw_string(10, 10, "Hello
   MaixPy", image.Color.from_rgb(255,
   0, 0))
```

这里的参数依次是：`x`，`y`，`text`，`color`，`x` 和 `y` 是文字的左上角坐标，`text` 是要写的文字，`color` 是文字的颜色，可以使用 `image.Color.from_rgb` 方法创建颜色。

还可以放大字体，传参 `scale` 即可：

```
1 | img.draw_string(10, 10, "Hello
   MaixPy", image.Color.from_rgb(255,
   0, 0), scale=2)
```

获取字体的宽度和高度：

```
1 | w, h = image.string_size("Hello
2 | MaixPy", scale=2)
   print(w, h)
```

注意这里 `scale` 是放大倍数，默认是 `1`，和 `draw_string` 应该保持一致。

十、中文支持和自定义字体

`image` 模块支持加载 `ttf/otf` 字体，默认字体只支持英文，如果要显示中文或者自定义字体可以先下载字体文件到设备上，然后加载字体。

系统也内置了几个字体，在 `/maixapp/share/font` 目录下面，代码示例：

```
1  from maix import image, display,
2  app, time
3
4  image.load_font("sourcehansans",
5  "/maixapp/share/font/SourceHanSansCN-
6  Regular.otf", size = 32)
7  print("fonts:", image.fonts())
8  image.set_default_font("sourcehansans")
9
10 disp = display.Display()
11
12 img = image.Image(disp.width(),
13 disp.height())
14 img.draw_string(2, 2, "你好! Hello,
    world!", image.Color.from_rgba(255,
    0, 0, 0.8))
    disp.show(img)
    while not app.need_exit():
        time.sleep(1)
```

加载字体文件，然后设置默认的字体，也可以不设置默认的字体，在写字的函数参数设置：

```
1  | img.draw_string(2, 2, "你好! Hello,
    world!", image.Color.from_rgba(255,
    0, 0, 0.8), font="sourcehansans")
```

注意 `string_size` 方法也会使用设置的默认字体计算大小，也可以通过 `font` 参数单独设置要计算大小的字体。

十一、画线

`image.Image` 提供了 `draw_line` 方法，可以在图像上画线：

```
1 | from maix import image
2 |
3 | img = image.Image(320, 240,
4 | image.Format.FMT_RGB888)
   img.draw_line(10, 10, 100, 100,
   image.Color.from_rgb(255, 0, 0))
```

这里的参数依次是：`x1`，`y1`，`x2`，`y2`，`color`，`x1` 和 `y1` 是线的起点坐标，`x2` 和 `y2` 是线的终点坐标，`color` 是线的颜色，可以使用 `image.Color.from_rgb` 方法创建颜色。

十二、画圆

`image.Image` 提供了 `draw_circle` 方法，可以在图像上画圆：

```
1 | from maix import image
2 |
3 | img = image.Image(320, 240,
4 | image.Format.FMT_RGB888)
   img.draw_circle(100, 100, 50,
   image.Color.from_rgb(255, 0, 0))
```

这里的参数依次是：`x`，`y`，`r`，`color`，`x` 和 `y` 是圆心坐标，`r` 是半径，`color` 是圆的颜色，可以使用 `image.Color.from_rgb` 方法创建颜色。

十三、缩放图像

`image.Image` 提供了 `resize` 方法，可以缩放图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.resize(160, 120)
  print(img, img_new)
```

注意这里 `resize` 方法返回一个新的图像对象，原图像不变。

十四、剪裁图像

`image.Image` 提供了 `crop` 方法，可以剪裁图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.crop(10, 10, 100, 100)
  print(img, img_new)
```

注意这里 `crop` 方法返回一个新的图像对象，原图像不变。

十五、旋转图像

`image.Image` 提供了 `rotate` 方法，可以旋转图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.rotate(90)
  print(img, img_new)
```

注意这里 `rotate` 方法返回一个新的图像对象，原图像不变。

十六、拷贝图像

`image.Image` 提供了 `copy` 方法，可以拷贝一份独立的图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.copy()
  print(img, img_new)
```

十七、仿射变换

`image.Image` 提供了 `affine` 方法，可以进行仿射变换，即提供当前图中三个及以上的点坐标，以及目标图中对应的点坐标，可以自动进行图像的旋转、缩放、平移等操作变换到目标图像：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img_new = img.affine([(10, 10),
  (100, 10), (10, 100)], [(10, 10),
  (100, 20), (20, 100)])
  print(img, img_new)
```

更多参数和用法请参考 API 文档。

十八、画关键点

`image.Image` 提供了 `draw_keypoints` 方法，可以在图像上画关键点：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5
6 keypoints = [10, 10, 100, 10, 10,
7              100]
8 img.draw_keypoints(keypoints,
9 image.Color.from_rgb(255, 0, 0),
10 size=10, thickness=1, fill=False)
```

在坐标 `(10, 10)`、`(100, 10)`、`(10, 100)` 画三个红色的关键点，关键点的大小是 `10`，线宽是 `1`，不填充。

十九、画十字

`image.Image` 提供了 `draw_cross` 方法，可以在图像上画十字：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img.draw_cross(100, 100,
6 image.Color.from_rgb(255, 0, 0),
7 size=5, thickness=1)
```

在坐标 `(100, 100)` 画一个红色的十字，十字的延长大小是 `5`，所以线段长度为 `2 * size + thickness`，线宽是 `1`。

二十、画箭头

`image.Image` 提供了 `draw_arrow` 方法，可以在图像上画箭头：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
img.draw_arrow(10, 10, 100, 100,
image.Color.from_rgb(255, 0, 0),
thickness=1)
```

在坐标 `(10, 10)` 画一个红色的箭头，箭头的终点是 `(100, 100)`，线宽是 `1`。

二十一、画图

`image.Image` 提供了 `draw_image` 方法，可以在图像上画图：

```
1 from maix import image
2
3 img = image.Image(320, 240,
4 image.Format.FMT_RGB888)
5 img2 = image.Image(100, 100,
6 image.Format.FMT_RGB888)
img2.draw_rect(10, 10, 90, 90,
image.Color.from_rgb(255, 0, 0))
img.draw_image(10, 10, img2)
```

二十二、转换格式

`image.Image` 提供了 `to_format` 方法，可以转换图像格式：

```
from maix import image
```

```
3   img = image.Image(320, 240,  
4   image.Format.FMT_RGB888)  
5   img_new =  
6   img.to_format(image.Format.FMT_BGR888)  
7   print(img, img_new)  
   img_jpg =  
   img.to_format(image.Format.FMT_JPEG)  
   print(img, img_new)
```

注意这里 `to_format` 方法返回一个新的图像对象，原图像不变

二十三、 maix.image.Image 对象和 Numpy/OpenCV 格式互相 转换

见 [MaixPy 使用 OpenCV 文档](#)

二十四、和 bytes 数据互相 转换

`image.Image` 提供了 `to_bytes` 方法，可以转换图像为 `bytes` 数据：

```
from maix import image  
  
img = image.Image(320, 240,  
image.Format.FMT_RGB888)  
data = img.to_bytes()  
print(type(data), len(data),  
img.data_size())  
  
img_jpeg = image.from_bytes(320,  
240, image.Format.FMT_RGB888, data)
```



```
print(img_jpeg)
img =
img_jpeg.to_format(image.Format.FMT_RGB)
print(img)
```

这里 `to_bytes` 获得一个新的 `bytes` 对象，是独立的内存，不会影响原图。

`image.Image` 构造函数中传入 `data` 参数可以直接从 `bytes` 数据构造图像对象，注意新的图像也是独立的内存，不会影响到 `data`。

因为涉及到内存拷贝，所以这个方法比较耗时，不建议频繁使用。

如果你想用不拷贝的方式优化程序（不建议轻易使用，写不好代码会导致程序容易崩溃，），请看 API 文档。

二十五、更多基础 API 使用方法

更多 API 使用方法请参考 `maix.image` 模块的文档。

MaixCAM MaixPy 使用 OpenCV

一、简介

对于 MaixCAM，因为使用了 Linux，并且性能基本能够支撑使用 `Python` 版本的 `OpenCV`，所以除了使用 `maix` 模块，你也可以直接使用 `cv2` 模块。

本文例程以及更多可以在

[MaixPy/examples/vision/opencv](#) 中找到。

注意 `OpenCV` 的函数基本都是 CPU 计算的，能使用 `maix` 的模块尽量不使用 `OpenCV`，因为 `maix` 有很多函数都是经过硬件加速过的。

二、`maix.image.Image` 对象和 `Numpy/OpenCV` 格式互相转换

`maix.image.Image` 对象可以转换成 `numpy` 数组，这样就能给 `numpy` 和 `opencv` 等库使用：

```
1 from maix import image, time,
2   display, app
3
4 disp = display.Display()
5
6 while not app.need_exit():
7     img = image.Image(320, 240,
```

```

8 image.Format.FMT_RGB888)
9     img.draw_rect(0, 0, 100, 100,
10 image.COLOR_RED, thickness=-1)
11     t = time.ticks_ms()
12     img_bgr = image.image2cv(img,
13 ensure_bgr=True, copy=True)
14     img2 = image.cv2image(img_bgr,
15 bgr=True, copy=True)
16     print("time:", time.ticks_ms() -
17 t)
18     print(type(img_bgr),
19 img_bgr.shape)
20     print(type(img2), img2)
21     print("")
22     disp.show(img2)

```

前面的程序因为每次转换都要拷贝一次内存，所以速度会比较慢，下面为优化速度版本，如果不是极限追求速度不建议使用，容易出错：

```

1 from maix import image, time,
2 display, app
3
4 disp = display.Display()
5
6 while not app.need_exit():
7     img = image.Image(320, 240,
8 image.Format.FMT_RGB888)
9     img.draw_rect(0, 0, 100, 100,
10 image.COLOR_RED, thickness=-1)
11
12     t = time.ticks_ms()
13     img_rgb = image.image2cv(img,
14 ensure_bgr=False, copy=False)
15     img2 = image.cv2image(img_rgb,
16 bgr=False, copy=False)
17     print("time:", time.ticks_ms() -
18 t)
19     print(type(img_rgb),
20 img_rgb.shape)
21     print(type(img2), img2)

```

```
disp.show(img2)
```

- `img_rgb = image.image2cv(img, ensure_bgr=False, copy=False)` 中 `img_rgb` 会直接使用 `img` 的数据，不会产生内存拷贝，注意此时得到的 `img_rgb` 是 `RGB` 图，`opencv` 的 API 都是认为图是 `BGR` 的，所以用 `opencv` 的 API 操作图像时要注意，如果你无法掌控请设置 `ensure_bgr` 为 `True`。
- `img2 = image.cv2image(img_rgb, bgr=False, copy=False)` 中设置了 `copy` 为 `False`，即直接使用 `img_rgb` 的内存，不会新拷贝一份内存，所以速度更快了，但是需要小心，在 `img2` 使用结束前 `img_bgr` 不能被销毁，否则程序会崩溃。
- 注意因为借用了内存，所以更改转换后的图像也会影响到转换前的图像。

三、加载一张图片

```
1 import cv2
2
3 file_path =
4 "/maixapp/share/icon/detector.png"
5 img = cv2.imread(file_path)
   print(img)
```

因为 `cv2` 模块比较臃肿，`import cv2` 可能会需要一点时间。

四、显示图像到屏幕

但是由于直接使用了官方的 OpenCV，没有对接显示，所以要显示到屏幕上需要转换成 `maix.image.Image` 对象后再用 `display` 来显示：

```
1  from maix import display, image,
2      time
3  import cv2
4
5  disp = display.Display()
6
7  file_path =
8  "/maixapp/share/icon/detector.png"
9  img = cv2.imread(file_path)
10
11  img_show = image.cv2image(img)
12  disp.show(img_show)
13
14  while not app.need_exit():
15      time.sleep(1)
```

五、使用 OpenCV 函数

以边缘检测为例：

基于上面的代码，使用 `cv2.Canny` 函数即可：

```
1  from maix import image, display,
2      app, time
3  import cv2
4
5  file_path =
6  "/maixapp/share/icon/detector.png"
7  img0 = cv2.imread(file_path)
8
9  disp = display.Display()
10
11  while not app.need_exit():
12      img = img0.copy()
13
14
```

```

15     # canny method
16     t = time.ticks_ms()
17     edged = cv2.Canny(img, 180, 60)
18     t2 = time.ticks_ms() - t
19
20     # show by maix.display
21     t = time.ticks_ms()
    img_show = image.cv2image(edged)
    print(f"edge time: {t2}ms,
convert time: {time.ticks_ms() -
t}ms")
    disp.show(img_show)

```

六、使用摄像头

在 PC 上，我们使用 `OpenCV` 的 `VideoCapture` 类来读取摄像头，对于 `MaixCAM`，`OpenCV` 没有适配，我们可以用 `maix.camera` 模块来读取摄像头，然后给 `OpenCV` 使用。

通过 `image.image2cv` 函数将 `maix.image.Image` 对象转为 `numpy.ndarray` 对象给 `OpenCV` 使用：

```

1  from maix import image, display,
2  app, time, camera
3  import cv2
4
5  disp = display.Display()
6  cam = camera.Camera(320, 240,
7  image.Format.FMT_BGR888)
8
9  while not app.need_exit():
10     img = cam.read()
11
12     # convert maix.image.Image
13     object to numpy.ndarray object
14     t = time.ticks_ms()
15     img = image.image2cv(img,
16     ensure_bgr=False, copy=False)
17

```

```

18     print("time: ", time.ticks_ms()
19         - t)
20
        # canny method
        edged = cv2.Canny(img, 180, 60)

        # show by maix.display
        img_show = image.cv2image(edged,
bgr=True, copy=False)
        disp.show(img_show)

```

七、读取 USB 摄像头

先在开发板设置里面 **USB设置** 中选择 **USB 模式** 为 **HOST** 模式。如果没有屏幕，可以用

`examples/tools/maixcam_switch_usb_mode.py` 脚本进行设置。

```

1  from maix import image, display, app
2  import cv2
3  import sys
4
5  cap = cv2.VideoCapture(0)
6  cap.set(cv2.CAP_PROP_FRAME_WIDTH,
7  640)
8  cap.set(cv2.CAP_PROP_FRAME_HEIGHT,
9  480)
10 # cap.set(cv2.CAP_PROP_CONVERT_RGB,
11 0)
12
13 disp = display.Display()
14
15 if not cap.isOpened():
16     print("无法打开摄像头")
17     sys.exit(1)
18 print("开始读取")
19 while not app.need_exit():
20     ret, frame = cap.read()
21     if not ret:
22         print("无法读取帧")

```

```
        break
    img = image.cv2image(frame,
bgr=True, copy=False)
    disp.show(img)
```



MaixCAM MaixPy 寻找直线

2024-05-09

▼ 更新历史

日期	版本	作者	更新内容
2024-05-09	1.0.0	lxowalle	初版文档

阅读本文前，确保已经知晓如何开发MaixCAM，详情请阅读 [快速开始](#)

一、简介

在视觉应用中，在巡迹小车、巡线机器人等应用中经常需要寻找线条的功能。本文将介绍：

- 如何使用MaixPy来实现巡线功能
- 如何使用MaixCam的默认应用程序巡线

二、如何使用MaixPy来寻找直线

MaixPy的 `maix.image.Image` 中提供了 `get_regression` 方法来寻找直线

2.1. 代码示例

一个简单的示例，实现寻找并画出直线

```
1  from maix import camera, display,
2  image
3
4  cam = camera.Camera(320, 240)
5  disp = display.Display()
6
7  # thresholds = [[0, 80, 40, 80, 10,
8  80]]          # red
9  thresholds = [[0, 80, -120, -10, 0,
10 30]]          # green
11 # thresholds = [[0, 80, 30, 100,
12 -120, -60]]   # blue
13
14 while 1:
15     img = cam.read()
16
17     lines =
18     img.get_regression(thresholds,
19 area_threshold = 100)
20     for a in lines:
21         img.draw_line(a.x1(),
22 a.y1(), a.x2(), a.y2(),
23 image.COLOR_GREEN, 2)
24         theta = a.theta()
25         rho = a.rho()
26         if theta > 90:
27             theta = 270 - theta
28         else:
29             theta = 90 - theta
30         img.draw_string(0, 0,
31 "theta: " + str(theta) + ", rho: " +
32 str(rho), image.COLOR_BLUE)
33
34     disp.show(img)
```

步骤：

1. 导入image、camera、display模块

```
1 | from maix import image, camera,  
   display
```

2. 初始化摄像头和显示

```
1 | cam = camera.Camera(320, 240) #  
2 | 初始化摄像头，输出分辨率320x240  
   RGB格式  
   disp = display.Display()
```

3. 从摄像头获取图片并显示

```
1 | while 1:  
2 |     img = cam.read()  
3 |     disp.show(img)
```

4. 调用 `get_regression` 方法寻找摄像头图片中的直线，并画到屏幕上

```
1 | lines =  
2 | img.get_regression(thresholds,  
3 | area_threshold = 100)  
4 | for a in lines:  
5 |     img.draw_line(a.x1(),  
6 | a.y1(), a.x2(), a.y2(),  
7 | image.COLOR_GREEN, 2)  
8 |     theta = a.theta()  
9 |     rho = a.rho()  
10 |    if theta > 90:  
        theta = 270 - theta  
    else:  
        theta = 90 - theta  
    img.draw_string(0, 0,  
"theta: " + str(theta) + ",  
rho: " + str(rho),  
image.COLOR_BLUE)
```

- `img` 是通过 `cam.read()` 读取到的摄像头图像，当初初始化的方式为 `cam = camera.Camera(320, 240)` 时，`img` 对象是一张分辨率为320x240的RGB图。
- `img.get_regression` 用来寻找直线，`thresholds` 是一个颜色阈值列表，每个元素是一个颜色阈值，同时找到多个阈值就传入多个，每个颜色阈值的格式为 `[L_MIN, L_MAX, A_MIN, A_MAX, B_MIN, B_MAX]`，这里的 `L`、`A`、`B` 是 LAB 颜色空间的三个通道，`L` 通道是亮度，`A` 通道是红绿通道，`B` 通道是蓝黄通道。`pixels_threshold` 是一个像素面积的阈值，用来过滤一些不需要直线。
- `for a in lines` 用来遍历返回的 `Line` 对象，其中 `a` 就是当前的 `Line` 对象。通常 `get_regression` 函数只会返回一个 `Line` 对象，如果需要寻找多条直线，可以尝试使用 `find_line` 方法
- 使用 `img.draw_line` 来画出找到的线条，`a.x1()`，`a.y1()`，`a.x2()`，`a.y2()` 分别代表直线两端的坐标
- 使用 `img.draw_string` 在左上角显示直线与x轴的夹角，`a.theta()` 是直线与y轴的夹角，这里为了方便理解转换成直线与x轴的夹角 `theta`，`a.rho()` 是原点与直线的垂线的长度。

5. 通过maixvision运行代码，就可以寻线啦，看看效果吧



2.2. 常用参数说明

列举常用参数说明，如果没有找到可以实现应用的参数，则需要考虑是否使用其他算法实现，或者基于目前算法的结果扩展所需的功能

参数	说明
thresholds	基于lab颜色空间的阈值，thresholds=[l_min, l_max, a_min, a_max, b_min, b_max]分别表示： 亮度范围为[l_min, l_max] 绿色到红色的分量范围为[a_min, a_max] 蓝色到黄色的分量范围为[b_min, b_max] 可同时设置多个阈值
invert	使能阈值反转， 使能后传入阈值与实际阈值相反，即thresholds=[l_max, l_min, a_max, a_min, b_max, b_min]
roi	设置算法计算的矩形区域，roi=[x1, y1, x2, y2]，x1, y1表示矩形区域左上角坐标，x2, y2表示矩形区域的右下角坐标，默认roi=[0, 0, 1000, 1000]

参数	说明
area_threshold	过滤像素面积小于area_threshold 单位为像素点，默认为10。 该参数可用于过滤一些无用的小直线。
pixels_threshold	过滤有效像素点小于pixels_threshold 默认为10。 该参数可用于过滤一些无用的小直线。

本文介绍常用方法，更多 API 请看 API 文档的 [image](#) 部分。

2.3. 提升巡线的速度

这里提供几个提升巡线速度的方法

1. 选择合适的分辨率

越大的分辨率计算速度越慢，可以根据识别距离和精度的要求来选择更合适的分辨率

2. 使用灰度图识别

使用灰度图识别时，算法只会处理一个通道，有更快的识别速度，在颜色单一的环境会很有用。注意此时向 `get_regression` 传入 `thresholds` 时，只有 `l_min` 和 `l_max` 有效。

获取灰度图的方法：

```
# 方法1
cam = camera.Camera(320, 240,
                    image.Format.FMT_GRAYSCALE)
# MaixPy v4.2.1后支持
gray_img = cam.read()
# 获取灰度图
```

```
8 | # 方法2
    cam = camera.Camera(320, 240)
    img = cam.read()
    gray_img =
    img.to_format(image.Format.FMT_GRAY)
    # 获取灰度图
```

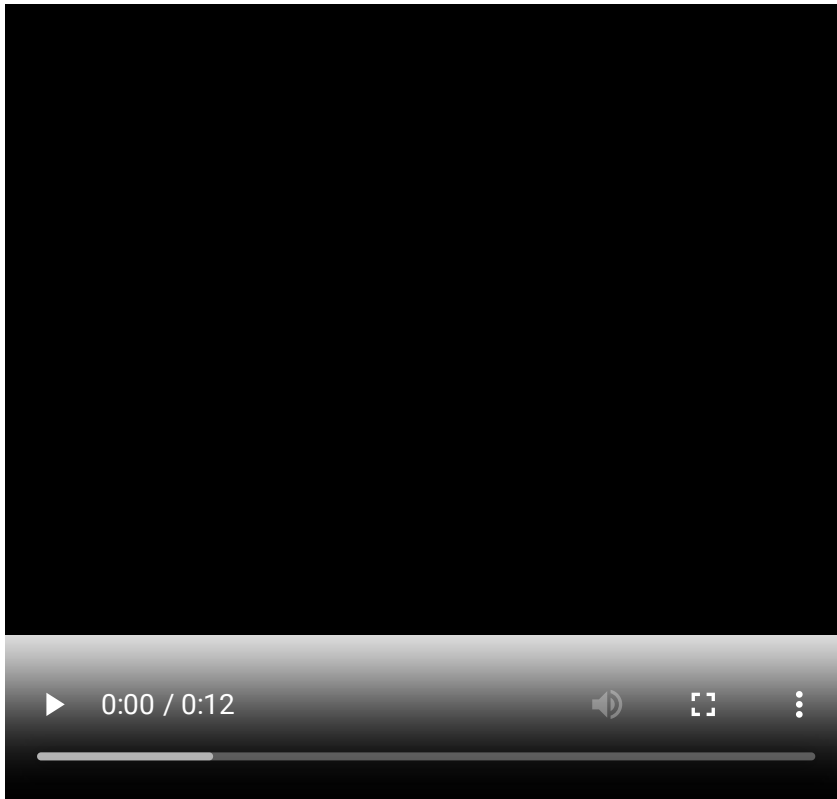
三、如何使用MaixCam的默认应用程序寻找直线

为了快速验证寻找直线的功能，可以先使用MaixCam提供的 `line_tracking` 应用程序来验证寻找直线的效果。

3.1. 使用方法

1. 选择并打开 `Line tracking` 应用
2. 点击屏幕中需要识别的直线，左侧会显示该直线的颜色
3. 点击左侧（界面中 `L A B` 下方的颜色）需要检测的颜色
4. 此时就可以识别到对应的直线了，同时串口也会输出直线的坐标和角度信息。

3.2. 演示



3.3. 进阶操作

3.3.1. 手动设置LAB阈值寻找直线

APP提供手动设置LAB阈值来精确的寻找直线

操作方法：

1. 点击 左下角 选项图标 ， 进入配置模式
2. 将 摄像头对准 需要 寻找的物体 ， 点击 屏幕上的 目标 直线 ， 此时界面中 L A B 下方会显示该物体对应颜色的 矩形框 ， 并显示该物体颜色的 LAB值 。
3. 点击下方选项 L Min, L Max, A Min, A Max, B Min, B Max ， 点击后右侧会出现滑动条来设置该选

项值。这些值分别对应LAB颜色格式的L通道、A通道和B通道的最小值和最大值

4. 参考步骤2计算的物体颜色的 LAB值，将 L Min, L Max, A Min, A Max, B Min, B Max 调整到合适的值，即可识别到对应的直线。

例如 LAB=(20, 50, 80)，由于 L=20，为了适配一定范围让 L Min=10，L Max=30；同理，由于 A=50，让 A Min=40，A Max=60；由于 B=80，让 B Min=70，B Max=90。

3.3.2. 通过串口协议获取检测数据

寻找直线应用支持通过串口（默认波特率为115200）上报检测到的直线信息。

由于上报信息只有一条，这里直接用示例来说明上报信息的内容。

例如上报信息为：

```
1 | AA CA AC BB 0E 00 00 00 E1 09 FC 01  
   | 01 00 E9 01 6F 01 57 00 C1 C6
```

- AA CA AC BB：协议头部，内容固定
- 0E 00 00 00：数据长度，除了协议头部和数据长度外的总长度，这里表示长度为14
- E1：标志位，用来标识串口消息标志
- 09：命令类型，对于寻找直线APP应用该值固定为0x09
- FC 01 01 00 E9 01 6F 01 57 00：直线的两端坐标和角度信息，每个值用小端格式的2字节表示。 FC

01 和 01 00 表示第一个端点坐标为(508, 1), E9

01 和 6F 01 表示第二个端点坐标为(489, 367), 57

00 表示直线与x轴的角度为87度

- C1 C6 : CRC 校验值, 用以校验帧数据在传输过程中是否出错



MaixCAM MaixPy 寻找色块

2024-04-03

▼ 更新历史

日期	版本	作者	更新内容
2024-04-03	1.0.0	neucrack	初版文档
2024-04-03	1.0.1	lxowalle	添加寻找色块的详细用

阅读本文前，确保已经知晓如何开发MaixCAM，详情请阅读 [快速开始](#)

一、简介

本文将介绍如何使用MaixPy来寻找色块，以及如何使用MaixCam的默认应用程序寻找色块。

在视觉应用中，寻找色块是一个非常常见的需求，比如机器人找色块，自动化生产线找色块等等，即需要识别画面中的特定的颜色区域，获取这个区域的位置和大小等信息。

二、使用 MaixPy 寻找色块

MaixPy的 `maix.image.Image` 中提供了 `find_blobs` 方法，可以方便的找色块。

2.1. 如何寻找色块

一个简单的示例，实现寻找色块并画框

```
1  from maix import image, camera,
2  display
3
4  cam = camera.Camera(320, 240)
5  disp = display.Display()
6
7  # 根据色块颜色选择对应配置
8  thresholds = [[0, 80, 40, 80, 10,
9  80]]          # red
10 # thresholds = [[0, 80, -120, -10,
11 0, 30]]        # green
12 # thresholds = [[0, 80, 30, 100,
13 -120, -60]]    # blue
14
15 while 1:
16     img = cam.read()
17     blobs =
18     img.find_blobs(thresholds,
19     pixels_threshold=500)
20     for blob in blobs:
21         img.draw_rect(blob[0],
22         blob[1], blob[2], blob[3],
23         image.COLOR_GREEN)
24     disp.show(img)
```

步骤：

1. 导入image、camera、display模块

```
1 | from maix import image, camera,
   display
```

2. 初始化摄像头和显示

```

1 | cam = camera.Camera(320, 240) #
2 | 初始化摄像头，输出分辨率320x240
   | RGB格式
   | disp = display.Display()

```

3. 从摄像头获取图片并显示

```

1 | while 1:
2 |     img = cam.read()
3 |     disp.show(img)

```

4. 调用 `find_blobs` 方法寻找摄像头图片中的色块，并画到屏幕上

```

1 | blobs =
2 | img.find_blobs(thresholds,
3 | pixels_threshold=500)
   | for blob in blobs:
   |     img.draw_rect(blob[0],
   |                  blob[1], blob[2], blob[3],
   |                  image.COLOR_GREEN)

```

- `img` 是通过 `cam.read()` 读取到的摄像头图像，当初初始化的方式为 `cam = camera.Camera(320, 240)` 时，`img` 对象是一张分辨率为320x240的RGB图。
- `img.find_blobs` 用来寻找色块，`thresholds` 是一个颜色阈值列表，每个元素是一个颜色阈值，同时找到多个阈值就传入多个，每个颜色阈值的格式为 `[L_MIN, L_MAX, A_MIN, A_MAX, B_MIN, B_MAX]`，这里的 `L`、`A`、`B` 是 `LAB` 颜色空间的三个通道，`L` 通道是亮度，`A` 通道是红绿通道，`B` 通道是蓝黄通道。`pixels_threshold` 是一个像素点数量的阈值，用来过滤一些不需要的小色块。

- o `img.draw_rect` 用来画色块框, `blob[0]`、`blob[1]`、`blob[2]`、`blob[3]` 分别代表色块左上角坐标x, 色块左上角坐标y, 色块宽度w和色块高度h

2.2. 常用参数说明

列举常用参数说明, 如果没有找到可以实现应用的参数, 则需要考虑是否使用其他算法实现, 或者基于目前算法的结果扩展所需的功能

参数	说明
thresholds	基于lab颜色空间的阈值, thresholds_l_max, a_min, a_max, b_min, b_max分别表示: 亮度范围为[l_min, l_max] 绿色到红色的分量范围为[a_min, a_max] 蓝色到黄色的分量范围为[b_min, b_max] 可同时设置多个阈值
invert	使能阈值反转, 使能后传入阈值与实际阈值相反, 即阈值取反
roi	设置算法计算的矩形区域, roi=[x1, y1, x2, y2] x1, y1表示矩形区域左上角坐标, w, h表示矩形区域的宽度和高度, 默认值为None
area_threshold	过滤像素面积小于area_threshold 单位为像素点, 默认为10。 该参数可用于过滤一些无用的小色块
pixels_threshold	过滤有效像素点小于pixels_threshold 默认为10。 该参数可用于过滤一些无用的小色块

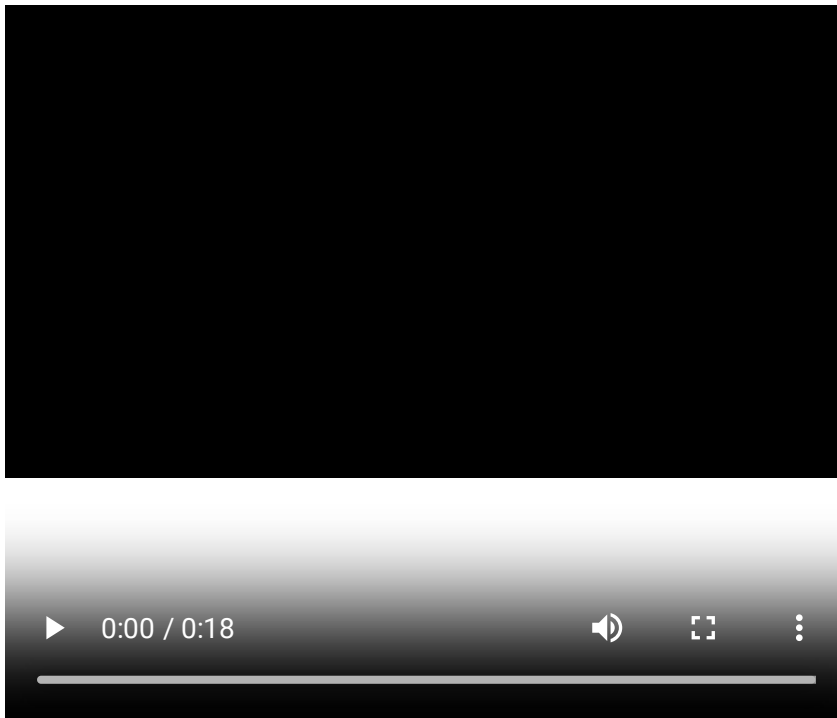
本文介绍常用方法，更多 API 请看 API 文档的 [image](#) 部分。

三、离线设置阈值

为了快速验证寻找色块的功能，可以先使用MaixCam提供的寻找色块应用程序来体验寻找色块的效果。

3.1. 演示

打开设备，选择 [找色块](#) 应用，然后在下方选择要识别的颜色，或者自定义颜色，即可以识别到对应的颜色了，在下方 [设置栏](#) 会显示当前设置的 [阈值范围](#)，同时串口也会输出识别到的坐标和颜色信息。

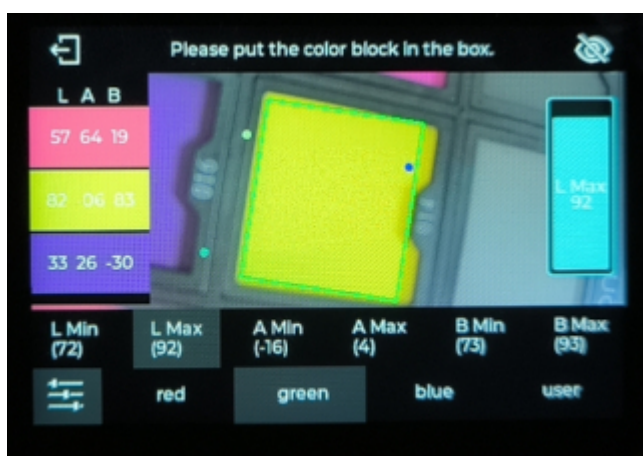


[源码地址](#)

3.2. 快速使用

3.2.1. 使用默认阈值

寻找色块APP提供了 red、green、blue、user 四种配置，其中 red、green 和 blue 用来寻找 红色、绿色和蓝色 的色块，user 自定义的阈值在程序退出时会保存下来，下次打开应用时会加载上一次调试的阈值。快速体验时通过 点击 界面下方 按钮 即可切换到对应配置，APP 界面参考如下：



3.2.2. 快速调试阈值

操作方法：

1. 将 摄像头对准 需要 寻找的物体， 点击 屏幕上的 目标 物体，此时 左侧 会显示该物体对应颜色的 矩形框，并显示该物体颜色的LAB值。
2. 点击出现的 矩形框，系统将会 自动设置 LAB阈值，此时画面将会画出该物体边缘。

这个方法优点是方便，快捷，可以很快的设置阈值并找到对应的色块。缺点是还不够精确，可以在下一步中手动微调。

3.2.3. 手动微调阈值

操作方法：

1. 点击 左下角 选项图标 ， 进入配置模式
2. 将 摄像头对准 需要 寻找的物体 ， 点击 屏幕上的 目标 物体 ， 此时 左侧 会显示该物体对应颜色的 矩形框 ， 并显示该物体颜色的 LAB值 。
3. 点击下方选项 L Min, L Max, A Min, A Max, B Min, B Max ， 点击后右侧会出现滑动条来设置该选项值。这些值分别对应LAB颜色格式的L通道、A通道和B通道的最小值和最大值
4. 参考步骤2计算的物体颜色的 LAB值 ， 将 L Min, L Max, A Min, A Max, B Min, B Max 调整到合适的值，即可识别到对应的色块。

例如 LAB=(20, 50, 80) ， 由于 L=20 ， 为了适配一定范围让 L Min=10 ， L Max=30 ;同理， 由于 A=50 ， 让 A Min=40 ， A Max=60 ; 由于 B=80 ， 让 B Min=70 ， B Max=90 。

这个方法可以更精确的找到合适的阈值，搭配 快速调试阈值 的方法，轻松找到想要的阈值。

3.2.4. 通过串口协议获取识别结果

寻找色块APP支持通过串口（默认波特率为115200）上报检测到的色块信息。

由于上报信息只有一条，这里直接用示例来说明上报信息的内容。

例如上报信息为：

```
1 | AA CA AC BB 14 00 00 00 E1 08 EE 00
   37 00 15 01 F7 FF 4E 01 19 00 27 01
```

- **AA CA AC BB** : 协议头部, 内容固定
- **14 00 00 00** : 数据长度, 除了协议头部和数据长度外的总长度
- **E1** : 标志位, 用来标识串口消息标志
- **08** : 命令类型, 对于寻找色块APP应用该值固定为 0x08
- **EE 00 37 00 15 01 F7 FF 4E 01 19 00 27 01 5A 00** : 已找到色块的四个顶点坐标, 每个值用小端格式的2字节表示。**EE 00** 和 **37 00** 表示第一个顶点坐标为(238, 55), **15 01** 和 **F7 FF** 表示第二个顶点坐标为(277, -9), **4E 01** 和 **19 00** 表示第三个顶点坐标为(334, 25), **27 01** 和 **5A 00** 表示第四个顶点坐标为(295, 90)。
- **A7 20** : CRC 校验值, 用以校验帧数据在传输过程中是否出错

四、关于LAB颜色空间

LAB颜色空间和RGB颜色空间一样是一种表示颜色的方法, LAB可以表示人眼能看到的所有颜色。如果需要了解LAB可以去网络上搜索相关文章, 那样更详细, 而对于你应该只需要了解为什么选用LAB对于MaixPy的优势。

LAB对于MaixPy的优势:

1. LAB颜色空间的色域比RGB都要大, 因此完全可以替换RGB。
2. LAB颜色空间下, 由于L通道是亮度通道, 我们常常设置到较大的范围即可 (常用[0,80]) , 而编写代码

时主要关注是A通道和B通道，这样可以减少大量的时间在纠结颜色阈值如何选择的问题上。

3. LAB颜色空间的色感知更均匀，更容易用代码调试。例如，对于只需要寻找红色色块，可以固定L通道和B通道值，只需要调整A通道的值即可（这是在颜色精度要求不高的情况下）；如果是RGB通道则基本需要R、G、B三个通道同时变动才能找到合适的阈值。



MaixCAM MaixPy 二维码识别

2024-04-03

▼ 更新历史

日期	版本	作者	更新内容
2024-04-03	1.0.0	lxowalle	初版文档

阅读本文前，确保已经知晓如何开发MaixCAM，详情请阅读 [快速开始](#)

一、简介

本文介绍如何使用MaixPy来识别二维码

二、使用 MaixPy 识别二维码

MaixPy的 `maix.image.Image` 中提供了 `find_qrcodes` 方法，用来识别二维码。

2.1. 如何识别二维码

一个简单的示例，实现识别二维码并画框

```
1 from maix import image, camera,  
2 display  
3
```

```

4 | cam = camera.Camera(320, 240)
5 | disp = display.Display()
6 |
7 | while 1:
8 |     img = cam.read()
9 |     qrcodes = img.find_qrcodes()
10 |    for qr in qrcodes:
11 |        corners = qr.corners()
12 |        for i in range(4):
13 |            img.draw_line(corners[i]
14 | [0], corners[i][1], corners[(i + 1)
% 4][0], corners[(i + 1) % 4][1],
image.COLOR_RED)
            img.draw_string(qr.x(),
qr.y() - 15, qr.payload(),
image.COLOR_RED)
            disp.show(img)

```

步骤:

1. 导入image、camera、display模块

```

1 | from maix import image, camera,
   display

```

2. 初始化摄像头和显示

```

1 | cam = camera.Camera(320, 240) #
2 | 初始化摄像头，输出分辨率320x240
   RGB格式
   disp = display.Display()

```

3. 从摄像头获取图片并显示

```

1 | while 1:
2 |     img = cam.read()
3 |     disp.show(img)

```

4. 调用 `find_qrcodes` 方法识别摄像头中的二维码

```
1 | qrcodes = img.find_qrcodes()
```

- `img` 是通过 `cam.read()` 读取到的摄像头图像，当初初始化的方式为 `cam = camera.Camera(320, 240)` 时，`img` 对象是一张分辨率为320x240的RGB图。
- `img.find_qrcodes` 用来寻找二维码，并将查询结果保存到 `qrcodes`，以供后续处理

5. 处理识别二维码的结果并显示到屏幕上

```
1 | for qr in qrcodes:  
2 |     corners = qr.corners()  
3 |     for i in range(4):  
4 |  
5 |         img.draw_line(corners[i][0],  
                        corners[i][1], corners[(i + 1)  
                        % 4][0], corners[(i + 1) % 4]  
                        [1], image.COLOR_RED)  
        img.draw_string(qr.x(),  
                        qr.y() - 15, qr.payload(),  
                        image.COLOR_RED)
```

- `qrcodes` 是通过 `img.find_qrcodes()` 查询二维码的结果，如果找不到二维码则 `qrcodes` 内部为空
- `qr.corners()` 用来获取已扫描到的二维码的四个顶点坐标，`img.draw_line()` 利用这四个顶点坐标画出二维码的形状
- `img.draw_string` 用来显示二维码的内容和位置等信息，其中 `qr.x()` 和 `qr.y()` 用来获取二维码左上角坐标x和坐标y，`qr.payload()` 用来获取二维码的内容

2.2. 常用参数说明

列举常用参数说明，如果没有找到可以实现应用的参数，则需要考虑是否使用其他算法实现，或者基于目前算法的结果扩展所需的功能

参数	
roi	设置算法计算的矩形区域，roi=[x, y,
qrcoder_type	设置二维码库解码器类型， 可以选择image.QRCodeDecoderType.QR_CODE_DECODER_TYPE_ZBAR在MaixPy v4.7.7及以后的版本有效。但识别精度相对较低。默认使用QR_CODE_DECODER_TYPE_ZBAR。

本文介绍常用方法，更多 API 请看 API 文档的 [image](#) 部分。

三、使用硬件加速的方法识别二维码

MaixPy内置了一个 `image.QRCodeDetector` 对象可以使用硬件加速的方法识别二维码，在320x224分辨率下单帧算法最高速度可以到60+fps

注：MaixPy v4.7.8之后的版本支持该方法（不包括v4.7.8）

3.1. 使用方法

```
1 from maix import camera, display,
2 app, image
3
4 cam = camera.Camera(320, 224)
```

```

5   disp = display.Display()
6   detector = image.QRCodeDetector()
7
8   while not app.need_exit():
9       img = cam.read()
10
11       qrcodes = detector.detect(img)
12       for q in qrcodes:
13           img.draw_string(0, 0,
14 "payload: " + q.payload(),
15 image.COLOR_BLUE)
16
17       disp.show(img)

```

步骤:

1. 导入image、camera、display模块

```

1 | from maix import camera,
   display, app, image

```

2. 捕获和显示图像

```

1 | cam = camera.Camera(320, 224)
2 | disp = display.Display()
3
4 | while not app.need_exit():
5 |     img = cam.read()
6 |     disp.show(img)

```

- 创建 `Camera` 和 `Display` 对象，通过
`cam.read()` 方法来捕获图像，用
`disp.show()` 方法来显示图像

3. 创建 `QRCodeDetector` 对象来检测二维码

```

1 | detector =
   image.QRCodeDetector()

```


4. 使用 `detect` 方法来检测二维码，检测结果保存到 `qrcores` 变量中

```
1 | qrcores = detector.detect(img)
2 | for q in qrcores:
3 |     img.draw_string(0, 0,
        "payload: " + q.payload(),
        image.COLOR_BLUE)
```

- 注意：检测过程中会占用NPU资源，如果此时有其他模型也再使用，则可能导致意外的结果
- 检测的结果与 `find_qrcores` 返回结果的数据结构一致，参考 `QRCode` 对象的方法来获取检测结果。例如：调用 `q.payload()` 即可获取二维码的内容字符串。



MaixCAM MaixPy 识别 Apriltag 标签

2024-04-03

▼ 更新历史

日期	版本	作者	更新内容
2024-04-03	1.0.0	lxowalle	初版文档

阅读本文前，确保已经知晓如何开发MaixCAM，详情请阅读 [快速开始](#)

一、简介

本文介绍如何使用MaixPy来识别Apriltag标签

二、使用 MaixPy 识别 Apriltag标签

MaixPy的 `maix.image.Image` 中提供了 `find_apriltags` 方法，可以识别apriltag标签。

2.1. 如何识别Apriltag标签

一个简单的示例，实现识别apriltag标签并画框

```
1 from maix import image, camera,  
2 display  
3
```

```

4  cam = camera.Camera()
5  disp = display.Display()
6
7  families =
8  image.ApriltagFamilies.TAG36H11
9  x_scale = cam.width() / 160
10 y_scale = cam.height() / 120
11
12 while 1:
13     img = cam.read()
14
15     new_img = img.resize(160, 120)
16     apriltags =
17 new_img.find_apriltags(families =
18 families)
19     for a in apriltags:
20         corners = a.corners()
21
22         for i in range(4):
23             corners[i][0] =
24 int(corners[i][0] * x_scale)
25             corners[i][1] =
26 int(corners[i][1] * y_scale)
27             x = int(a.x() * x_scale)
28             y = int(a.y() * y_scale)
29             w = int(a.w() * x_scale)
30             h = int(a.h() * y_scale)
31
32             for i in range(4):
33                 img.draw_line(corners[i]
34 [0], corners[i][1], corners[(i + 1)
35 % 4][0], corners[(i + 1) % 4][1],
36 image.COLOR_RED)
37                 img.draw_string(x + w, y,
38 "id: " + str(a.id()),
39 image.COLOR_RED)
40                 img.draw_string(x + w, y +
41 15, "family: " + str(a.family()),
42 image.COLOR_RED)
43
44         disp.show(img)

```

步骤:

1. 导入image、camera、display模块

```
1 | from maix import image, camera,  
   | display
```

2. 初始化摄像头和显示

```
1 | cam = camera.Camera()  
2 | disp = display.Display()
```

3. 从摄像头获取图片并显示

```
1 | while 1:  
2 |     img = cam.read()  
3 |     disp.show(img)
```

4. 调用 `find_apriltags` 方法识别摄像头图片中的 apriltag标签

```
1 | new_img = img.resize(160, 120)  
2 | apriltags =  
   | new_img.find_apriltags(families  
   | = families)
```

- `img` 是通过 `cam.read()` 读取到的摄像头图像
- `img.resize(160, 120)` 是用来将图像缩放得更小，用更小的图像来让算法计算得更快
- `new_img.find_apriltags(families = families)` 用来寻找apriltag标签，并将查询结果保存到 `apriltags`，以供后续处理。其中 `families` 用来选择apriltag族，默认为 `image.ApriltagFamilies.TAG36H11`

5. 处理识别标签的结果并显示到屏幕上

```

1  for a in apriltags:
2      # 获取位置信息（并映射坐标到原
3      图）
4      x = int(a.x() * x_scale)
5      y = int(a.y() * y_scale)
6      w = int(a.w() * x_scale)
7      corners = a.corners()
8      for i in range(4):
9          corners[i][0] =
10     int(corners[i][0] * x_scale)
11          corners[i][1] =
12     int(corners[i][1] * y_scale)
13
14     # 显示
15     for i in range(4):
16
17         img.draw_line(corners[i][0],
18                       corners[i][1], corners[(i + 1)
19 % 4][0], corners[(i + 1) % 4]
20 [1], image.COLOR_RED)
21         img.draw_string(x + w,
22 y, "id: " + str(a.id()),
23 image.COLOR_RED)
24         img.draw_string(x + w,
25 y + 15, "family: " +
26 str(a.family()),
27 image.COLOR_RED)
28         img.draw_string(x + w,
29 y + 30, "rotation : " + str(180
30 * a.rotation() // 3.1415),
31 image.COLOR_RED)

```

- 遍历 `apriltags` 的成员，`apriltags` 是通过 `img.find_apriltags()` 扫描 `apriltag` 标签的结果，如果找不到标签则 `apriltags` 的成员为空
- `x_scale` 和 `y_scale` 用来映射坐标，由于 `new_img` 是缩放后的图像，计算 `apriltag` 的坐标时需要经过映射后才能正常的画在原图 `img` 上

- `a.corners()` 用来获取已扫描到的标签的四个顶点坐标, `img.draw_line()` 利用这四个顶点坐标画出标签的形状
- `img.draw_string` 用来显示标签的内容, 其中 `a.x()` 和 `a.y()` 用来获取标签左上角坐标x和坐标y, `a.id()` 用来获取标签的id, `a.family()` 用来获取标签族类型, `a.rotation()` 用来获取标签的旋转角度。

2.2. 常用参数说明

列举常用参数说明, 如果没有找到可以实现应用的参数, 则需要考虑是否使用其他算法实现, 或者基于目前算法的结果扩展所需的功能

参数	说明	
roi	设置算法计算的矩形区域, roi=[x, y, w, h], x, y表示矩形区域左上角坐标, w, h表示矩形区域的宽度和高度, 默认为整张图片	计算坐标 宽和高为 <code>img.fin</code> <code>100, 100</code>
families	apriltag标签家族类型	扫描TAG <code>img.fin</code> <code>image.Ap</code>

本文介绍常用方法, 更多 API 请看 API 文档的 [image](#) 部分。

2.3. 测距1：物体垂直与摄像头的距离

这里提供一种使用 $\text{distance} = k / \text{width}$ 的公式来测距, 其中 distance 是摄像头和物体的距离, 单位 mm , k 是一个常量, width 是物体在画面中的宽度, 单位是像素点.

测量方法分两步: 1. 测量常量系数 k ; 2. 通过常量系数和标签宽度来计算物体与摄像头的距离

2.3.1. 前期准备

1. `apriltag` 标签纸
2. 尺子(或其他测距工具)

2.3.2. 测量常量系数 k

- 将 `apriltag` 标签纸固定, 并在距离 `apriltag` 标签 20cm 处固定 `maixcam`
- 使用 `maixcam` 检测 `apriltag` 标签并计算标签的宽度, 参考代码:

```
1  from maix import camera,
2  display
3  import math
4
5  '''
6  x1,y1,x2,y2: apriltag宽度的两点坐标, 一般通过corners()方法获取
7  返回标签的宽度, 单位为像素点
8  '''
9
10 def caculate_width(x1, y1, x2,
11                    y2):
12     return math.sqrt((x2 -
13                      x1)**2 + (y2 - y1)**2)
14
15 cam = camera.Camera(160, 120)
16 disp = display.Display()
17
18 while 1:
19     img = cam.read()
```

```

20
21     apriltags =
22 img.find_apriltags()
23     for a in apriltags:
24         corners = a.corners()
25
        # 通过水平方向的两个坐标点
        计算宽度
        width =
        caculate_width(corners[0][0],
        corners[0][1], corners[1][0],
        corners[1][1])
        # 打印apriltag标签的实际
        宽度
        print(f'apriltag width:
        {width}')
        disp.show(img)

```

- 计算常量系数k

```

1  '''
2  width: 当距离为distance时,检测到
3  apriltag标签的宽度
4  distance: 检测apriltag标签时距离
5  apriltag标签的实际距离, 单位mm
6  返回常量系数
7  '''
8  def caculate_k(width,
9  distance):
10     return width * distance

    # 距离200mm时检测到标签宽度为43个像
    素
    k = caculate_k(43, 200)

```

2.3.3. 通过常量系数计算摄像头和物体间的距离

```

1  '''
2  width: apriltag标签的宽度

```



```

3 k: 常量系数
4 返回摄像头与物体的距离,单位mm
5 '''
6 def caculate_distance(width, k):
7     return k / width
8
9 distance = caculate_distance(55,
    8600)

```

2.3.4. 完整的代码参考:

```

1 from maix import camera, display,
2 image
3 import math
4
5 '''
6 x1,y1,x2,y2: apriltag宽度的两点坐标,
7 一般通过corners()方法获取
8 返回标签的宽度,单位为像素点
9 '''
10 def caculate_width(x1, y1, x2, y2):
11     return math.sqrt((x2 - x1)**2 +
12 (y2 - y1)**2)
13
14 '''
15 width: 当距离为distance时,检测到
16 apriltag标签的宽度
17 distance: 检测apriltag标签时距离
18 apriltag标签的实际距离, 单位mm
19 返回常量系数
20 '''
21 def caculate_k(width, distance):
22     return width * distance
23
24 '''
25 width: apriltag标签的宽度
26 k: 常量系数
27 返回摄像头与物体的距离,单位mm
28 '''
29 def caculate_distance(width, k):
30     return k / width
31

```

```

32
33
34 cam = camera.Camera(192, 108)
35 disp = display.Display()
36
37 # 距离200mm时检测到标签宽度为43个像素
38 k = caculate_k(43, 200)
39
40 while 1:
41     img = cam.read()
42
43     apriltags = img.find_apriltags()
44     for a in apriltags:
45         corners = a.corners()
46         for i in range(4):
47             img.draw_line(corners[i]
48 [0], corners[i][1], corners[(i + 1)
49 % 4][0], corners[(i + 1) % 4][1],
50 image.COLOR_GREEN)
51
52         # 通过水平方向的两个坐标点计算
        宽度
        width =
        caculate_width(corners[0][0],
        corners[0][1], corners[1][0],
        corners[1][1])

        # 计算距离
        distance =
        caculate_distance(width, k)

        print(f'apriltag width:
        {width} distance:{distance} mm')

        disp.show(img)

```

上面的方法是通过 `apriltag` 的宽度计算距离, 同样也可以扩展为使用高度来计算距离. 但需要注意该方法是在对距离估测, 实际应用中会有些许误差存在.

2.4. 测距2：利用apriltag标签测量物体到摄像头的距离

通过 `apriltag` 标签测距，可以比较准确的测量标签在空间中的位置，这里我们也利用 `find_apriltag()` 方法返回的参数来计算任意位置的 `apriltag` 标签到摄像头的位置，当然前提是你必须检测到 `apriltag` 标签。

测量方法共两步：1. 计算常量系数k。2.通过 `find_apriltag()` 返回的位置信息计算标签到摄像头的距离

优点：可以在 `apriltag` 标签旋转、与摄像头有偏移的情况测量出标签到摄像头的距离

2.4.1. 前期准备

1. `apriltag` 标签纸
2. 尺子(或其他测距工具)

2.4.2. 测量常量系数

- 将 `apriltag` 标签纸固定,并在距离 `apriltag` 标签 20cm处固定 `maixcam`
- 使用 `maixcam` 检测 `apriltag` 标签并通过 `z_translation` 计算常量系数, 参考代码:

```
1  from maix import camera,  
2  display  
3  
4  '''  
5  z_trans: 当距离为distance时,检测  
6  到apriltag标签z_translation()的值  
7  distance: 检测apriltag标签时距离  
8  apriltag标签的实际距离, 单位mm  
9  返回常量系数
```

```

10     '''
11     def caculate_k(z_trans,
12     distance):
13         return distance / z_trans
14
15     cam = camera.Camera(160, 120)
16     disp = display.Display()
17
18     while 1:
19         img = cam.read()
20
21         apriltags =
img.find_apriltags()
        for a in apriltags:
            k =
caculate_k(a.z_translation(),
200)
            print(f"k:{k}")
            disp.show(img)

```

2.4.3. 测量标签到物体的距离

- 通过 `apriltag` 标签返回的

`x_translation`, `y_translation` 和

`z_translation` 计算标签到摄像头的距离

```

1     '''
2     x_trans: 检测apriltag标签返回的
3     x_translation()的值
4     y_trans: 检测apriltag标签返回的
5     y_translation()的值
6     z_trans: 检测apriltag标签返回的
7     z_translation()的值
8     k: 常量系数
9     返回距离, 单位mm
10    '''
11
12    def calculate_distance(x_trans,
13    y_trans, z_trans, k):
14        return k *
15        math.sqrt(x_trans * x_trans +
16        y_trans * y_trans + z_trans *

```

```
z_trans)
```

2.4.4. 完整的代码参考:

```
1  from maix import camera, display,
2  image
3  import math
4
5  '''
6  z_trans: 当距离为distance时,检测到
7  apriltag标签z_translation()的值
8  distance: 检测apriltag标签时距离
9  apriltag标签的实际距离, 单位mm
10  返回常量系数
11  '''
12  def caculate_k(z_trans, distance):
13      return distance / z_trans
14
15  '''
16  x_trans: 检测apriltag标签返回的
17  x_translation()的值
18  y_trans: 检测apriltag标签返回的
19  y_translation()的值
20  z_trans: 检测apriltag标签返回的
21  z_translation()的值
22  k: 常量系数
23  返回距离, 单位mm
24  '''
25  def calculate_distance(x_trans,
26  y_trans, z_trans, k):
27      return abs(k * math.sqrt(x_trans
28  * x_trans + y_trans * y_trans +
29  z_trans * z_trans))
30
31  cam = camera.Camera(160, 120)
32  disp = display.Display()
33
34  # 距离200mm时检测到apriltag标签返回的
35  z_translation()为-9.7
36  k = caculate_k(-9.7, 200)
37
38
```

```

39 while 1:
40     img = cam.read()
41
42     apriltags = img.find_apriltags()
43     for a in apriltags:
44         corners = a.corners()
         for i in range(4):
             img.draw_line(corners[i]
[0], corners[i][1], corners[(i + 1)
% 4][0], corners[(i + 1) % 4][1],
image.COLOR_GREEN)

         # 计算距离
         x_trans = a.x_translation()
         y_trans = a.y_translation()
         z_trans = a.z_translation()
         distance =
calculate_distance(x_trans, y_trans,
z_trans, k)
         print(f'apriltag k:{k}
distance:{distance} mm')

disp.show(img)

```

这段代码使用 **MaixCAM** 不断读取摄像头图像，检测 **apriltag**，并通过常量系数以及标签在三个轴向的位移计算标签到摄像头的距离。计算出的距离以毫米为单位打印出来，需要注意该方法仍是在对距离估测，实际应用中会有些许误差存在。

MaixCAM MaixPy 条形码识别

2024-12-16

▼ 更新历史

日期	版本	作者	更新内容
2024-12-16	1.0.0	lxowalle	初版文档

阅读本文前，确保已经知晓如何开发MaixCAM，详情请阅读 [快速开始](#)

一、简介

本文介绍如何使用MaixPy来识别条形码

二、使用 MaixPy 识别条形码

MaixPy的 `maix.image.Image` 中提供了 `find_barcodes` 方法，用来识别条形码

2.1. 如何识别条形码

一个简单的示例，实现识别条形码并画框

```
1 from maix import image, camera,  
2 display  
3
```

```

4 | cam = camera.Camera(480, 320)
5 | disp = display.Display()
6 |
7 | while 1:
8 |     img = cam.read()
9 |
10 |     barcodes = img.find_barcodes()
11 |     for b in barcodes:
12 |         rect = b.rect()
13 |         img.draw_rect(rect[0],
14 | rect[1], rect[2], rect[3],
15 | image.COLOR_BLUE, 2)
        img.draw_string(0, 0,
        "payload: " + b.payload(),
        image.COLOR_GREEN)

        disp.show(img)

```

步骤:

1. 导入image、camera、display模块

```

1 | from maix import image, camera,
   display

```

2. 初始化摄像头和显示

```

1 | cam = camera.Camera(480, 320) #
2 | 初始化摄像头，输出分辨率480x320
   RGB格式
   disp = display.Display()

```

3. 从摄像头获取图片并显示

```

1 | while 1:
2 |     img = cam.read()
3 |     disp.show(img)

```

4. 调用 `find_barcodes` 方法识别摄像头中的条形码


```
1 | barcodes = img.find_barcodes()
```

- `img` 是通过 `cam.read()` 读取到的摄像头图像, 当初初始化的方式为 `cam = camera.Camera(320, 240)` 时, `img` 对象是一张分辨率为480x320的RGB图。
- `img.find_barcodes` 用来寻找条形码, 并将查询结果保存到 `barcodes`, 以供后续处理
- 注意: 条形码的间距较小, 并且一般宽度远大于高度, 所以在调整识别率和识别速度时可以尽量让识别目标图像的宽度更大, 高度更小

5. 处理识别条形码的结果并显示到屏幕上

```
1 | for b in barcodes:  
2 |     rect = b.rect()  
3 |     img.draw_rect(rect[0],  
4 | rect[1], rect[2], rect[3],  
    image.COLOR_BLUE, 2)  
    img.draw_string(0, 0,  
    "payload: " + b.payload(),  
    image.COLOR_GREEN)
```

- `barcodes` 是通过 `img.find_barcodes()` 查询条形码的结果, 如果找不到条形码则 `barcodes` 内部为空
- `b.rect()` 用来获取已扫描到的条形码的位置和大小, `img.draw_rect()` 利用这些位置信息画出条形码的形状
- `img.draw_string` 用来显示条形码的内容和位置等信息, `b.payload()` 用来获取条形码的内容

2.2. 常用参数说明

列举常用参数说明，如果没有找到可以实现应用的参数，则需要考虑是否使用其他算法实现，或者基于目前算法的结果扩展所需的功能

参数	说明	示
roi	设置算法计算的矩形区域， roi=[x, y, w, h], x, y表示矩形区域左上角坐标， w, h表示矩形区域的宽度和高度， 默认为整张图片	计算坐标为(5 宽和高为100 <code>img.find_b</code> <code>[50, 50, 10</code>

本文介绍常用方法，更多 API 请看 API 文档的 [image](#) 部分。